

Proyecto Final - Tecnología de Base de Datos I

Proyecto Final — Migración Completa MariaDB → PostgreSQL 18 (Base employees)

Campo	Descripción
Universidad	Universidad Privada Domingo Savio — Facultad de Ingeniería
Asignatura	Tecnología de Base de Datos I — Bloque 3: Migración de un sistema informático a otro SGBD
Tema	Proyecto final: migración completa de datos y estructura (MariaDB → PostgreSQL), vistas y respaldo
Estudiante	Marco Antonio Kiataque Uchima
Docente	Jared Lopez Leños
Ubicación	Santa Cruz – Bolivia
Fecha de entrega	29 de septiembre de 2026
Entorno	Linux (Mininux / Ubuntu) + Docker Compose — postgres:18 , mariadb:11.8.9-ubi9 , pgloader 3.6.10, Adminer, pgAdmin4

Archivos entregables de este proyecto:

- 1. Proyecto_Final_Informe.md — este informe en Markdown.
- 2. Proyecto_Final_Informe.pdf — versión en PDF del mismo informe.
- 3. docker-compose.yml — entorno Docker del laboratorio.
- 4. migracion.load — script de migración pgloader.
- 5. ac06.sh — script de vistas, verificación y backup.
- 6. employees_diagrama.png + employees_postgres.sql — diseño PgModeler (Actividad 5, base de este proyecto).
- 7. pdb_employees_backup.dump — backup binario final (pg_dump -F c -b , ~36 MB).
- 8. README.md — portada del repositorio GitHub.
- 9. Repositorio GitHub: https://github.com/MarcoKiataque27/tecnologia_de_base_de_datos_1.git

Toda la evidencia es salida de comandos en texto plano, sin capturas de pantalla.

Índice

- 1. [Objetivo y escenario](#)
- 2. [Fase 1: entorno Docker Compose](#)
- 3. [Fase 2: carga de la BD origen en MariaDB](#)
- 4. [Fase 3: análisis pre-migración](#)
- 5. [Fase 4: migración de datos con pgloader](#)
- 6. [Fase 5: verificación de integridad en PostgreSQL](#)
- 7. [Fase 6: migración de vistas](#)
- 8. [Fase 7: respaldo y restauración \(pg_dump / pg_restore \)](#)
- 9. [Fase 8: script automatizado ac06.sh](#)
- 10. [Incidencias y solución](#)
- 11. [Conclusiones](#)
- 12. [Eje transversal y ODS](#)
- 13. [Repositorio GitHub](#)
- 14. [Referencias](#)

1. Objetivo y escenario

Migrar la base `employees` (6 tablas + 2 vistas, ~3.9 millones de filas) desde MariaDB 11.8.9 hacia PostgreSQL 18, garantizando:

- Estructura (DDL) compatible (Act. 5: `pdb_employees` , tipos, PK/FK).
- Datos íntegros (conteos y CRC idénticos origen/destino).
- Vistas recreadas (`current_dept_emp` , `dept_emp_latest_date`).
- Respaldo binario final restaurable.

Elemento	Detalle
Origen	MariaDB 11.8.9 (contenedor <code>mariadb</code> , <code>3306:3306</code>), BD <code>employees</code> , usuario <code>root</code> / <code>123456</code> y <code>marco</code> / <code>123123</code> , BD inicial <code>mdb_marco</code>
Destino	PostgreSQL 18 (contenedor <code>postgresql</code> , <code>5432:5432</code> , <code>TZ=America/La_Paz</code>), BD base <code>pdb_marco</code> , BD migrada <code>employees</code> + BD final <code>pdb_employees</code> , usuario <code>marco</code> / <code>123123</code>
Herramientas	Docker Compose, <code>mariadb-dump</code> , <code>pgloader 3.6.10</code> , <code>psql</code> , <code>pg_dump</code> / <code>pg_restore</code> , Adminer (<code>8081->8080</code>), pgAdmin4 (<code>8080->80</code>)
Dump origen	<code>test_db-master.tar.gz</code> → <code>employees.sql</code> + <code>test_employees_sha.sql</code> (validación CRC)

2. Fase 1: entorno Docker Compose

Lo que se hace en esta fase es levantar todo el laboratorio con un solo archivo, para no instalar los motores en la máquina física. Se aplica Docker Compose con cuatro servicios aislados pero en la misma red: PostgreSQL 18, MariaDB 11.8.9, Adminer y pgAdmin4. Esto garantiza que el experimento sea repetible: cualquier compañero que use el mismo `docker-compose.yml` obtiene los mismos puertos, usuarios y versiones.

2.1 `docker-compose.yml` real del estudiante

```
services:
  postgres:
    image: postgres:18
    container_name: postgresql
    restart: unless-stopped
    environment:
      POSTGRES_DB: pdb_marco
      POSTGRES_USER: marco
      POSTGRES_PASSWORD: 123123
      TZ: 'America/La_Paz'
      PGTZ: 'America/La_Paz'
    command: ["postgres", "-c", "timezone=America/La_Paz"]
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U marco -d pdb_marco"]
      interval: 10s
      timeout: 5s
      retries: 5
  mariadb:
    image: mariadb:11.8.9-ubi9
    container_name: mariadb
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: 123456
      MARIADB_DATABASE: mdb_marco
```

```

    MARIADB_USER: marco
    MARIADB_PASSWORD: 123123
  volumes:
    - mariadb_data:/var/lib/mysql
  ports:
    - "3306:3306"
  adminer:
    image: adminer
    container_name: adminer
    restart: always
    ports:
      - "8081:8080"
  pgadmin:
    image: dpage/pgadmin4
    container_name: pgadmin4
    restart: always
    ports:
      - "8080:80"
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@upds.com
      PGADMIN_DEFAULT_PASSWORD: 123123
    volumes:
      - pgadmin-data:/var/lib/pgadmin
volumes:
  mariadb_data:
  postgres_data:
  pgadmin-data:

```

Lo que se está aplicando aquí: el servicio `postgres` crea por defecto la base `pdb_marco` con el usuario `marco` y fija la zona horaria a `America/La_Paz` para que las fechas se guarden coherentes con Bolivia; el `healthcheck` con `pg_isready` evita que los siguientes comandos se ejecuten antes de que la BD acepte conexiones. El servicio `mariadb` usa la imagen `11.8.9-ubi9` que pidió el docente, con clave `root` `123456` y base inicial `mdb_marco`. `adminer` (puerto 8081) y `pgadmin` (puerto 8080) son solo las interfaces gráficas para inspeccionar sin pelear con la terminal. Los volúmenes guardan los datos aunque se apaguen los contenedores.

2.2 Levantar el entorno desde cero

Aquí se parte de un entorno limpio para demostrar que la migración funciona desde cero y no depende de datos viejos. Lo que se hace es: descargar la imagen exacta de MariaDB, borrar contenedores y volúmenes anteriores con `down -v` y volver a crear todo con `up -d`. El `down -v` es destructivo a propósito (elimina los datos de prueba), y el `up -d` levanta los 4 servicios en segundo plano. La salida lo confirma con los `Started` y el ingreso a MariaDB que muestra `Server version: 11.8.9`.

```
docker pull mariadb:11.8.9-ubi9
```

```

11.8.9-ubi9: Pulling from library/mariadb
Digest: sha256:5916802c9ed98ab8315709228e785d4b050ac9a56a1e5c438a2734de5b18ac26
Status: Downloaded newer image for mariadb:11.8.9-ubi9

```

```

cd ~/tecBD1
docker compose down -v
docker compose up -d

```

```

[+] down 8/8
✓ Container adminer          Removed
✓ Container pgadmin4         Removed
✓ Container postgresql       Removed
✓ Container mariadb          Removed
✓ Volume tecbd1_mariadb_data Removed
✓ Volume tecbd1_postgres_data Removed
✓ Network tecbd1_default     Removed
✓ Volume tecbd1_pgadmin-data Removed

```

```
[+] up 8/8
✓ Network tecbd1_default      Created
✓ Volume tecbd1_postgres_data Created
✓ Volume tecbd1_mariadb_data  Created
✓ Volume tecbd1_pgadmin-data  Created
✓ Container postgresql        Started
✓ Container pgadmin4          Started
✓ Container adminer           Started
✓ Container mariadb           Started
```

```
mariadb -h 127.0.0.1 -u root -p
```

```
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 11.8.9-MariaDB MariaDB Server
MariaDB [(none)]> exit;
Bye
```

3. Fase 2: carga de la BD origen en MariaDB

En esta fase se carga la base de ejemplo `employees` (la clásica `test_db` de MySQL) dentro de MariaDB, porque sin datos reales no hay nada que migrar. Lo que se aplica es: descomprimir `test_db-master.tar.gz`, importar `employees.sql` con el cliente `mariadb` y luego correr `test_employees_sha.sql`, que es una prueba de integridad que compara cantidad de filas y checksum CRC tabla por tabla. Cuando la salida dice `records_match OK / crc_match ok` y `summary result CRC OK`, significa que el origen está sano y sirve como patrón de comparación para todo lo que sigue.

```
tar -xvf test_db-master.tar.gz
cd test_db-master/
mariadb -h 127.0.0.1 -u root -p < employees.sql
```

```
mariadb -h 127.0.0.1 -u root -p < test_employees_sha.sql
```

```
INFO
TESTING INSTALLATION
table_name expected_records expected_crc
departments 9 4b315afa0e35ca6649df897b958345bcb3d2b764
dept_emp 331603 d95ab9fe07df0865f592574b3b33b9c741d9fd1b
dept_manager 24 9687a7d6f93ca8847388a42a6d8d93982a841c6c
employees 300024 4d4aa689914d8fd41db7e45c2168e7dcb9697359
salaries 2844047 b5a1785c27d75e33a4173aaa22ccf41ebd7d4a9f
titles 443308 d12d5f746b88f07e69b9e36675b6067abb01b60e
table_name found_records found_crc
departments 9 4b315afa0e35ca6649df897b958345bcb3d2b764
dept_emp 331603 d95ab9fe07df0865f592574b3b33b9c741d9fd1b
dept_manager 24 9687a7d6f93ca8847388a42a6d8d93982a841c6c
employees 300024 4d4aa689914d8fd41db7e45c2168e7dcb9697359
salaries 2844047 b5a1785c27d75e33a4173aaa22ccf41ebd7d4a9f
titles 443308 d12d5f746b88f07e69b9e36675b6067abb01b60e
table_name records_match crc_match
departments OK ok
dept_emp OK ok
dept_manager OK ok
employees OK ok
salaries OK ok
titles OK ok
```

```
computation_time 00:00:14
summary result CRC OK count OK
```

Verificación de bases existentes:

```
mariadb -h 127.0.0.1 -u root -p -e "SHOW DATABASES;"
```

```
+-----+
| Database |
+-----+
| employees |
| information_schema |
| mdb_marco |
| mysql |
| performance_schema |
| sys |
+-----+
6 rows in set
```

4. Fase 3: análisis pre-migración

Antes de migrar se investiga qué hay que migrar: cuántas tablas, qué tamaño tienen, qué vistas existen y cómo es el DDL original. Esto se aplica consultando `information_schema.tables` y `information_schema.views` en MariaDB. El hallazgo clave es que además de las 6 tablas hay 2 vistas (`current_dept_emp` y `dept_emp_latest_date`) que no viajan solas con pgloader y habrá que recrearlas a mano en PostgreSQL. También se extrae solo la estructura con `mariadb-dump --no-data` para estudiar las diferencias de sintaxis sin mover los 135 MB de datos.

4.1 Tablas y tamaños en MariaDB

```
SELECT table_name, table_rows, data_length, index_length
FROM information_schema.tables
WHERE table_schema = 'employees'
ORDER BY data_length DESC;
```

4.2 Vistas en el origen

```
SELECT table_name FROM information_schema.views
WHERE table_schema = 'employees';
```

```
table_name
dept_emp_latest_date
current_dept_emp
```

Evidencia de tablas/vistas (`evidencia_mariadb.txt`):

```
Tables_in_employees
current_dept_emp
departments
dept_emp
dept_emp_latest_date
dept_manager
employees
salaries
titles
Field Type Null Key Default Extra
emp_no int(11) NO PRI NULL
```

```

birth_date date          NO      NULL
first_name varchar(14)   NO      NULL
last_name varchar(16)    NO      NULL
gender enum('M','F')     NO      NULL
hire_date date           NO      NULL

```

4.3 Extracción de solo estructura (DDL)

```

docker exec mariadb mariadb-dump -u root -p123456 --no-data employees > employees-estructura.sql
ls -lh employees-estructura.sql

```

```
-rw-r--r-- 1 marco marco 7,9K employees-estructura.sql
```

El archivo contiene los 6 `CREATE TABLE` con `ENGINE=InnoDB` `DEFAULT CHARSET=utf8mb4`, `int(11)`, `enum('M','F')`, backticks y las 2 vistas `current_dept_emp` / `dept_emp_latest_date` (ver `employees-estructura.sql` adjunto, 198 líneas).

Punto 1.1 de la consigna (exportar datos con `mysqldump` o `SELECT ... INTO OUTFILE`): en este proyecto la exportación de datos no usa dump intermedio porque la herramienta elegida para el punto 1.2 es **pgloader**, que lee directo de MariaDB (`mysql://root:123456@127.0.0.1:3306/employees`) y escribe directo en PostgreSQL. El `mariadb-dump --no-data` se usa para estudiar y documentar la estructura, y el `pg_dump -F c` final genera el backup exigido en el formato de entrega.

4.4 Archivo de migración pgloader (`migracion.load`)

Este archivo es el corazón de la migración automática: le dice a pgloader de dónde leer y dónde escribir. Lo que se aplica es: conexión origen `mysql://root:123456@127.0.0.1:3306/employees` y destino `postgresql://marco:123123@127.0.0.1:5432/employees`, con `include drop + create tables + create indexes` para que reconstruya todo, 8 workers en paralelo y lotes de 10 000 filas para ir rápido. En el `CAST` se decide algo importante: `datetime` → `timestampz` (convirtiendo fechas cero en NULL) y `enum` → `text`. Por eso después la columna `gender` aparece como `text` y `emp_no` como `bigint`: fue una decisión de compatibilidad, distinta del DDL manual de la Act. 5 donde se usó `gender_enum` e `integer`.

```

LOAD DATABASE
FROM mysql://root:123456@127.0.0.1:3306/employees
INTO postgresql://marco:123123@127.0.0.1:5432/employees

WITH include drop,
     create tables,
     create indexes,
     reset sequences,
     workers = 8, concurrency = 2,
     batch rows = 10000

SET maintenance_work_mem to '256MB',
    work_mem to '32MB'

CAST type datetime to timestampz drop default drop not null
    using zero-dates-to-null,
    type enum to text
;

```

Decisiones de casteo: `datetime` → `timestampz` (con `zero-dates-to-null`), `enum` → `text`. Por eso en la BD migrada vía pgloader `gender` queda `text` y `emp_no` `bigint`, a diferencia del DDL manual de la Act. 5 (`gender_enum`, `integer`).

Instalación previa:

```
sudo apt update && sudo apt install pgloader -y
```

5. Punto 1 — Migración de Tablas: migración de datos con pgloader

Corresponde al Punto 1 de la consigna (15 pts desarrollo / 40 pts rúbrica: exportar 1.1, importar 1.2, conteo 1.3).

5.1 Crear la BD destino

```
psql -h localhost -U marco -d pdb_marco -c "CREATE DATABASE employees;"
```

Contraseña para usuario marco:
CREATE DATABASE

5.2 Ejecutar la migración

Aquí se ejecuta `pgloader migracion.load` y se lee su reporte como acta de la migración. Lo que está haciendo pgloader por dentro es: leer metadatos (21 objetos), crear 12 tablas, copiar en paralelo con 8 hilos y al final reconstruir 9 índices, 6 claves primarias y 6 foráneas. La fila `Total import time` ✓ 3919015 134.9 MB significa éxito total: se movieron las 3 919 015 filas sin ningún error (`errors = 0` en todas las tablas). La tabla `salaries` es la más pesada (2.8M filas, 94 MB) y por eso tarda más (~12 s).

```
pgloader migracion.load
```

```
2026-09-26T18:54:35.040020Z LOG pgloader version "3.6.10~devel"
2026-09-26T18:54:35.048024Z LOG Data errors in '/tmp/pgloader/'
2026-09-26T18:54:35.048024Z LOG Parsing commands from file #P"/home/marco/tecBD1/migracion.load"
2026-09-26T18:54:35.412205Z LOG Migrating from #<MYSQL-CONNECTION mysql://root@127.0.0.1:3306/employees>
2026-09-26T18:54:35.412205Z LOG Migrating into #<PGSQL-CONNECTION pgsq://marco@127.0.0.1:5432/employees>
2026-09-26T18:54:50.895944Z LOG report summary reset
```

table name	errors	rows	bytes	total time
-----	-----	-----	-----	-----
fetch meta data	0	21		0.252s
Create Schemas	0	0		0.000s
Create SQL Types	0	0		0.008s
Create tables	0	12		0.084s
Set Table OIDs	0	6		0.008s
-----	-----	-----	-----	-----
employees.salaries	0	2844047	94.2 MB	12.282s
employees.titles	0	443308	16.9 MB	3.682s
employees.dept_emp	0	331603	10.7 MB	7.832s
employees.employees	0	300024	13.2 MB	6.219s
employees.departments	0	9	0.1 kB	0.032s
employees.dept_manager	0	24	0.8 kB	0.020s
-----	-----	-----	-----	-----
COPY Threads Completion	0	8		12.274s
Create Indexes	0	9		3.090s
Index Build Completion	0	9		1.709s
Reset Sequences	0	0		0.080s
Primary Keys	0	6		0.028s
Create Foreign Keys	0	6		0.716s
Create Triggers	0	0		0.004s
Install Comments	0	0		0.000s
-----	-----	-----	-----	-----
Total import time	✓	3919015	134.9 MB	17.901s

Resultado: **3 919 015 filas (134.9 MB), 0 errores**, 6 PK + 6 FK + 9 índices reconstruidos.

Comprobación inmediata:

```
psql -h localhost -U marco -d employees -c "SELECT count(*) FROM employees.employees;"
```

Contraseña para usuario marco:

```
count
-----
 300024
(1 fila)
```

6. Punto 3 (parte 1) — Verificación de integridad en PostgreSQL

Corresponde al Punto 3 de la consigna (25 pts desarrollo / 40 pts rúbrica). La comparativa MariaDB vs PostgreSQL (3.2) y el análisis (3.3) están en 6.1b–6.1c.

6.1 Conteo por tabla

Esta es la verificación que pide el docente: contar filas en destino y compararlas con el origen y con el CRC. Lo que se aplica es un `UNION ALL` de seis `COUNT(*)`. Si los números coinciden (300 024 empleados, 9 departamentos, 331 603 asignaciones, 24 gerencias, 2 844 047 salarios, 443 308 títulos), se demuestra que no se perdió ni se duplicó ningún registro durante la copia en paralelo.

```
psql -h 127.0.0.1 -U marco -d pdb_employees -c "
SELECT 'employees.employees' AS tabla, COUNT(*) FROM employees.employees
UNION ALL SELECT 'employees.departments', COUNT(*) FROM employees.departments
UNION ALL SELECT 'employees.dept_emp', COUNT(*) FROM employees.dept_emp
UNION ALL SELECT 'employees.dept_manager', COUNT(*) FROM employees.dept_manager
UNION ALL SELECT 'employees.salaries', COUNT(*) FROM employees.salaries
UNION ALL SELECT 'employees.titles', COUNT(*) FROM employees.titles;"
```

tabla	count
employees.employees	300024
employees.departments	9
employees.dept_emp	331603
employees.dept_manager	24
employees.salaries	2844047
employees.titles	443308

(6 filas)

Los 6 conteos coinciden con el origen y con el CRC (`test_employees_sha.sql`).

6.1b Comparativa origen (MariaDB) vs destino (PostgreSQL) — Punto 3.2 de la consigna

La consigna exige conteo en **ambos** SGBD y comparación. El lado MariaDB se obtuvo con consultas directas el 29/09/2026 (`final_conteos_mariadb.txt`); el lado PostgreSQL del `UNION ALL` anterior. Ambos coinciden además con el CRC del `test_employees_sha.sql` (`ok` en las 6 tablas):

```
mariadb -h 127.0.0.1 -u root -p123456 -D employees -e "SELECT 'departments' AS tabla, COUNT(*) AS filas FROM departments
UNION ALL SELECT 'dept_emp', COUNT(*) FROM dept_emp UNION ALL SELECT 'dept_manager', COUNT(*) FROM dept_manager
UNION ALL SELECT 'employees', COUNT(*) FROM employees UNION ALL SELECT 'salaries', COUNT(*) FROM salaries UNION
ALL SELECT 'titles', COUNT(*) FROM titles;" > final_conteos_mariadb.txt
```

tabla	filas
departments	9
dept_emp	331603
dept_manager	24

```
employees 300024
salaries   2844047
titles     443308
```

Tabla	MariaDB (origen, conteo directo)	PostgreSQL (destino, COUNT(*))	CRC origen	Coincide
departments	9	9	4b315afa ok	Sí
dept_emp	331603	331603	d95ab9fe ok	Sí
dept_manager	24	24	9687a7d6f ok	Sí
employees	300024	300024	4d4aa689 ok	Sí
salaries	2844047	2844047	b5a1785c ok	Sí
titles	443308	443308	d12d5f746 ok	Sí

Análisis (Punto 3.3): coinciden el 100% de los conteos y los 6 CRC del origen están ok. La única diferencia documentada es de tipos, no de datos: pgloader casteó `enum` → `text` (`gender`) e `int` → `bigint` (`emp_no`) según el `CAST` del `migracion.load`, mientras el DDL manual de la Act. 5 usa `gender_enum` e `integer`. No hay filas perdidas ni duplicadas.

6.1c Checksums MD5 comparativos (Punto 3.1 de la consigna)

Para comparar **contenido** y no solo cantidad, se calculó el MD5 del contenido ordenado por clave primaria en `departments` (9 filas) y `dept_manager` (24 filas), en ambos motores el 29/09/2026:

```
mariadb -h 127.0.0.1 -u root -p123456 -D employees -e "SET SESSION group_concat_max_len = 1000000; SELECT 'departments'
AS tabla, MD5(GROUP_CONCAT(CONCAT_WS('|', dept_no, dept_name) ORDER BY dept_no SEPARATOR ';;')) AS checksum
FROM departments UNION ALL SELECT 'dept_manager', MD5(GROUP_CONCAT(CONCAT_WS('|', emp_no, dept_no, from_date,
to_date) ORDER BY emp_no, dept_no SEPARATOR ';;')) FROM dept_manager;" > checksums_mariadb.txt

psql -h 127.0.0.1 -U marco -d pdb_employees -c "SELECT 'departments' AS tabla, MD5(STRING_AGG(CONCAT_WS('|', dept_no,
dept_name), ';;' ORDER BY dept_no)) AS checksum FROM employees.departments UNION ALL SELECT 'dept_manager',
MD5(STRING_AGG(CONCAT_WS('|', emp_no::text, dept_no, from_date::text, to_date::text), ';;' ORDER BY emp_no,
dept_no)) FROM employees.dept_manager;" > checksums_postgresql.txt
```

```
tabla checksum
departments fa8cbd703ad11e1f06b31874f4ae55b5
dept_manager 1e7bec3546440ceca470f44da37de99f
```

```
tabla | checksum
-----+-----
departments | fa8cbd703ad11e1f06b31874f4ae55b5
dept_manager | 1e7bec3546440ceca470f44da37de99f
(2 filas)
```

Tabla	Checksum MariaDB	Checksum PostgreSQL	Resultado
departments	fa8cbd703ad11e1f06b31874f4ae55b5	fa8cbd703ad11e1f06b31874f4ae55b5	Coincide
dept_manager	1e7bec3546440ceca470f44da37de99f	1e7bec3546440ceca470f44da37de99f	Coincide

Lo que se está aplicando: en MariaDB se concatena cada fila con `CONCAT_WS('|', ...)` ordenada por PK y se resume con `MD5(GROUP_CONCAT(...))`; en PostgreSQL lo equivalente es `MD5(STRING_AGG(...))`. Que ambos hashes sean idénticos demuestra que el **contenido** migró sin alteraciones, no solo el conteo.

6.2 Estructura y restricciones (\d)

Además del conteo se verifica que la estructura llegó con sus reglas: el `\d` muestra tipos, nulabilidad, clave primaria e integridad referencial. Lo que se observa es que `emp_no` quedó `bigint` y `gender` quedó `text` por el casteo de pgloader, y que las 4 tablas hijas referencian a `employees` con `ON DELETE CASCADE`. Esto significa que si se borra un empleado, sus salarios, títulos y asignaciones se borran en cascada, igual que en el origen.

```
psql -h 127.0.0.1 -U marco -d pdb_employees -c "\d employees.employees"
```

```

                Tabla «employees.employees»
Columna      |      Tipo      | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
emp_no       | bigint         |              | not null |
birth_date   | date           |              | not null |
first_name    | character varying(14) |              | not null |
last_name     | character varying(16) |              | not null |
gender        | text           |              | not null |
hire_date     | date           |              | not null |
Índices:
    "idx_16803_primary" PRIMARY KEY, btree (emp_no)
Referenciada por:
    TABLE "dept_emp" CONSTRAINT "dept_emp_ibfk_1" FOREIGN KEY (emp_no) REFERENCES employees(emp_no) ON UPDATE RESTRICT ON DELETE CASCADE
    TABLE "dept_manager" CONSTRAINT "dept_manager_ibfk_1" FOREIGN KEY (emp_no) REFERENCES employees(emp_no) ON UPDATE RESTRICT ON DELETE CASCADE
    TABLE "salaries" CONSTRAINT "salaries_ibfk_1" FOREIGN KEY (emp_no) REFERENCES employees(emp_no) ON UPDATE RESTRICT ON DELETE CASCADE
    TABLE "titles" CONSTRAINT "titles_ibfk_1" FOREIGN KEY (emp_no) REFERENCES employees(emp_no) ON UPDATE RESTRICT ON DELETE CASCADE

```

6.3 Registros huérfanos = 0 (ambos SGBD, 6 relaciones)

Esta verificación busca hijos sin padre en las 6 relaciones del modelo, en **ambos** motores (Punto 3.1/3.2 de la consigna). Lo que se aplica es un `LEFT JOIN` contra la tabla padre filtrando `WHERE padre.pk IS NULL`: cada fila devuelta sería un huérfano. Que todo dé `0` significa que las claves foráneas quedaron íntegras después de la copia masiva.

```

mariadb -h 127.0.0.1 -u root -p123456 -D employees -e "SELECT 'dept_emp -> employees' AS relacion, COUNT(*) AS huerfanos
FROM dept_emp d LEFT JOIN employees e ON d.emp_no = e.emp_no WHERE e.emp_no IS NULL UNION ALL SELECT 'dept_emp
-> departments', COUNT(*) FROM dept_emp d LEFT JOIN departments p ON d.dept_no = p.dept_no WHERE p.dept_no IS
NULL UNION ALL SELECT 'dept_manager -> employees', COUNT(*) FROM dept_manager d LEFT JOIN employees e ON
d.emp_no = e.emp_no WHERE e.emp_no IS NULL UNION ALL SELECT 'dept_manager -> departments', COUNT(*) FROM
dept_manager d LEFT JOIN departments p ON d.dept_no = p.dept_no WHERE p.dept_no IS NULL UNION ALL SELECT
'salaries -> employees', COUNT(*) FROM salaries s LEFT JOIN employees e ON s.emp_no = e.emp_no WHERE e.emp_no
IS NULL UNION ALL SELECT 'titles -> employees', COUNT(*) FROM titles t LEFT JOIN employees e ON t.emp_no =
e.emp_no WHERE e.emp_no IS NULL;" > huerfanos_mariadb.txt

```

```

relacion      huerfanos
dept_emp -> employees      0
dept_emp -> departments    0
dept_manager -> employees  0
dept_manager -> departments 0
salaries -> employees      0
titles -> employees        0

```

```

psql -h 127.0.0.1 -U marco -d pdb_employees -v ON_ERROR_STOP=1 -c "SET max_parallel_workers_per_gather = 0; SET work_mem
= '4MB';" -c "SELECT 'dept_emp -> employees' AS relacion, COUNT(*) AS huerfanos FROM employees.dept_emp d LEFT
JOIN employees.employees e ON d.emp_no = e.emp_no WHERE e.emp_no IS NULL UNION ALL SELECT 'dept_emp ->
departments', COUNT(*) FROM employees.dept_emp d LEFT JOIN employees.departments p ON d.dept_no = p.dept_no
WHERE p.dept_no IS NULL UNION ALL SELECT 'dept_manager -> employees', COUNT(*) FROM employees.dept_manager d
LEFT JOIN employees.employees e ON d.emp_no = e.emp_no WHERE e.emp_no IS NULL UNION ALL SELECT 'dept_manager ->
departments', COUNT(*) FROM employees.dept_manager d LEFT JOIN employees.departments p ON d.dept_no = p.dept_no
WHERE p.dept_no IS NULL UNION ALL SELECT 'salaries -> employees', COUNT(*) FROM employees.salaries s LEFT JOIN
employees.employees e ON s.emp_no = e.emp_no WHERE e.emp_no IS NULL UNION ALL SELECT 'titles -> employees',
COUNT(*) FROM employees.titles t LEFT JOIN employees.employees e ON t.emp_no = e.emp_no WHERE e.emp_no IS
NULL;" > huerfanos_postgresql.txt

```

```

SET
SET
relacion      | huerfanos
-----+-----

```

```

dept_emp -> employees      |      0
dept_emp -> departments    |      0
dept_manager -> employees  |      0
dept_manager -> departments |      0
salaries -> employees      |      0
titles -> employees        |      0
(6 filas)

```

Los dos `SET` previos en PostgreSQL son el workaround de memoria compartida documentado en Incidencias (desactivan el paralelismo y bajan `work_mem` solo para esa sesión, sin tocar los datos).

7. Punto 2 — Migración de Vistas

Corresponde al Punto 2 de la consigna (10 pts desarrollo / 20 pts rúbrica: extraer 2.1, adaptar 2.2, crear y probar 2.3).

Las vistas no se migran solas: pgloader copia tablas, pero la lógica de las vistas hay que reescribirla porque MariaDB usa sintaxis propietaria (`ALGORITHM=UNDEFINED` , `DEFINER=root@%` , `SQL SECURITY DEFINER` , backticks). Lo que se hace es extraer la definición original con `SHOW CREATE VIEW` , limpiarla y recrearla en PostgreSQL con `CREATE OR REPLACE VIEW` . La vista `current_dept_emp` filtra la asignación vigente (`to_date = '9999-01-01'`) y `dept_emp_latest_date` agrupa por empleado con `MAX(from_date)` . Se fija `search_path TO employees, public` porque pgloader creó el esquema `employees` .

7.1 Definición original en MariaDB

```

docker exec -it mariadb mariadb -u root -p123456 -D employees -e "
SHOW CREATE VIEW current_dept_emp;
SHOW CREATE VIEW dept_emp_latest_date;" > ~/tecBD1/vistas_mariadb.txt

```

Contenido (`vistas_mariadb.txt` , vista `current_dept_emp`):

```

| current_dept_emp | CREATE ALGORITHM=UNDEFINED DEFINER=`root`@`%` SQL SECURITY DEFINER VIEW `current_dept_emp` AS
select `l`.`emp_no` AS `emp_no`,`d`.`dept_no` AS `dept_no`,`l`.`from_date` AS `from_date`,`l`.`to_date` AS `to_date`
from (`dept_emp` `d` join `dept_emp_latest_date` `l` on(`d`.`emp_no` = `l`.`emp_no` and `d`.`from_date` =
`l`.`from_date` and `l`.`to_date` = `d`.`to_date`)) | utf8mb3 | utf8mb3_uca1400_ai_ci |

```

Definición original de `dept_emp_latest_date` extraída el 29/09/2026 (`vista_latest_mariadb.txt` , Punto 2.1 de la consigna):

```

docker exec mariadb mariadb -u root -p123456 -D employees -e "SHOW CREATE VIEW dept_emp_latest_date;" >
vista_latest_mariadb.txt

```

```

View      Create View character_set_client  collation_connection
dept_emp_latest_date  CREATE ALGORITHM=UNDEFINED DEFINER=`root`@`%` SQL SECURITY DEFINER VIEW `dept_emp_latest_date`
AS select `dept_emp`.`emp_no` AS `emp_no`,max(`dept_emp`.`from_date`) AS `from_date`,max(`dept_emp`.`to_date`) AS
`to_date` from `dept_emp` group by `dept_emp`.`emp_no` utf8mb3 utf8mb3_uca1400_ai_ci

```

Definiciones finales en el dump:

```

VIEW `current_dept_emp` AS
select `l`.`emp_no`,`d`.`dept_no`,`l`.`from_date`,`l`.`to_date`
from (`dept_emp` `d` join `dept_emp_latest_date` `l`
on(`d`.`emp_no` = `l`.`emp_no` and `d`.`from_date` = `l`.`from_date` and `l`.`to_date` = `d`.`to_date`));

VIEW `dept_emp_latest_date` AS
select `dept_emp`.`emp_no`, max(`dept_emp`.`from_date`) AS `from_date`, max(`dept_emp`.`to_date`) AS `to_date`
from `dept_emp` group by `dept_emp`.`emp_no`;

```

7.2 Recreación en PostgreSQL

```
psql -h 127.0.0.1 -U marco -d pdb_employees -c "
SET search_path TO employees, public;

CREATE OR REPLACE VIEW current_dept_emp AS
SELECT l.emp_no, d.dept_no, l.from_date, l.to_date
FROM dept_emp l
JOIN departments d ON l.dept_no = d.dept_no
WHERE l.to_date = '9999-01-01';

CREATE OR REPLACE VIEW dept_emp_latest_date AS
SELECT emp_no, MAX(from_date) AS from_date, MAX(to_date) AS to_date
FROM dept_emp
GROUP BY emp_no;"
```

Contraseña para usuario marco:

```
SET
CREATE VIEW
CREATE VIEW
```

Adaptaciones aplicadas: se eliminaron `ALGORITHM=UNDEFINED`, `DEFINER=root@%`, `SQL SECURITY DEFINER` y backticks (propios de MariaDB); se fijó `search_path TO employees, public` porque pgloader crea el esquema `employees`.

7.3 Prueba de funcionamiento de las vistas (Punto 2.3 de la consigna)

Consultas de prueba ejecutadas el 29/09/2026 contra el esquema `employees` (donde viven los datos migrados):

```
psql -h 127.0.0.1 -U marco -d pdb_employees -c "SELECT COUNT(*) AS filas_current FROM employees.current_dept_emp; SELECT
* FROM employees.current_dept_emp LIMIT 10;" > prueba_vista_current.txt
psql -h 127.0.0.1 -U marco -d pdb_employees -c "SELECT COUNT(*) AS filas_latest FROM employees.dept_emp_latest_date;
SELECT * FROM employees.dept_emp_latest_date LIMIT 10;" > prueba_vista_latest.txt
```

filas_current

```
-----
      240124
(1 fila)

 emp_no | dept_no | from_date | to_date
-----+-----+-----+-----
  10001 | d005    | 1986-06-26 | 9999-01-01
  10002 | d007    | 1996-08-03 | 9999-01-01
  10003 | d004    | 1995-12-03 | 9999-01-01
  10004 | d004    | 1986-12-01 | 9999-01-01
  10005 | d003    | 1989-09-12 | 9999-01-01
  10006 | d005    | 1990-08-05 | 9999-01-01
  10007 | d008    | 1989-02-10 | 9999-01-01
  10009 | d006    | 1985-02-18 | 9999-01-01
  10010 | d006    | 2000-06-26 | 9999-01-01
  10012 | d005    | 1992-12-18 | 9999-01-01
(10 filas)
```

filas_latest

```
-----
      300024
(1 fila)

 emp_no | from_date | to_date
-----+-----+-----
  10001 | 1986-06-26 | 9999-01-01
  10002 | 1996-08-03 | 9999-01-01
```

```

10003 | 1995-12-03 | 9999-01-01
10004 | 1986-12-01 | 9999-01-01
10005 | 1989-09-12 | 9999-01-01
10006 | 1990-08-05 | 9999-01-01
10007 | 1989-02-10 | 9999-01-01
10008 | 1998-03-11 | 2000-07-31
10009 | 1985-02-18 | 9999-01-01
10010 | 2000-06-26 | 9999-01-01
(10 filas)

```

Lectura del resultado: ambas vistas **devuelven datos reales** en PostgreSQL, con lo que el Punto 2.3 queda probado. La diferencia de conteos es semántica y esperada: `dept_emp_latest_date` agrupa un registro por empleado (300 024, uno por cada empleado), mientras `current_dept_emp` filtra solo asignaciones vigentes (`to_date = '9999-01-01'`), es decir 240 124 empleados con departamento actual. (Nota: una primera prueba contra el esquema `public` —que solo contenía el DDL vacío de la Act. 5— devolvió 0 filas; al apuntar al esquema `employees` con `search_path` aparecen los datos. Se documenta el tropiezo porque demuestra entender los esquemas.)

8. Fase 7: respaldo y restauración (`pg_dump` / `pg_restore`)

Aquí se demuestra que la base migrada es respaldable y recuperable, que es lo que el inge pide como "backup adjunto". Lo que se aplica es: `pg_dump -F c -b` genera un respaldo binario comprimido (formato custom, con blobs), `pg_restore` lo devuelve creando esquema, tablas, datos, constraints e índices en ese orden, y un segundo `pg_dump` deja el respaldo final `pdb_employees_backup.dump` (~36 MB) con el esquema `employees` (datos de pgloader) más el esquema `public` (DDL de la Act. 5) y las 2 vistas. La salida `creando SCHEMA / creando TABLE / procesando datos` confirma que la restauración reconstruye todo sin errores.

8.1 Backup de la BD migrada por pgloader

```
pg_dump -h 127.0.0.1 -U marco -d employees -F c -b -v -f ~/tecBD1/temp_employees.dump
```

```

Contraseña:
pg_dump: salvando codificaciones = UTF8
pg_dump: extrayendo el contenido de la tabla «employees.departments»
pg_dump: extrayendo el contenido de la tabla «employees.dept_emp»
pg_dump: extrayendo el contenido de la tabla «employees.dept_manager»
pg_dump: extrayendo el contenido de la tabla «employees.employees»
pg_dump: extrayendo el contenido de la tabla «employees.salaries»
pg_dump: extrayendo el contenido de la tabla «employees.titles»

```

Archivo: `temp_employees.dump` (~35.9 MB).

8.2 Restauración en la BD final `pdb_employees`

```
pg_restore -h 127.0.0.1 -U marco -d pdb_employees -v ~/tecBD1/temp_employees.dump
```

```

Contraseña:
pg_restore: conectando a la base de datos para reestablecimiento
pg_restore: creando SCHEMA «employees»
pg_restore: creando TABLE «employees.departments»
pg_restore: creando TABLE «employees.dept_emp»
pg_restore: creando TABLE «employees.dept_manager»
pg_restore: creando TABLE «employees.employees»
pg_restore: creando TABLE «employees.salaries»
pg_restore: creando TABLE «employees.titles»
pg_restore: procesando datos de la tabla «employees.departments»
pg_restore: procesando datos de la tabla «employees.dept_emp»
pg_restore: procesando datos de la tabla «employees.dept_manager»

```

```

pg_restore: procesando datos de la tabla «employees.employees»
pg_restore: procesando datos de la tabla «employees.salaries»
pg_restore: procesando datos de la tabla «employees.titles»
pg_restore: creando CONSTRAINT «employees.departments idx_16877_primary»
pg_restore: creando CONSTRAINT «employees.dept_emp idx_16882_primary»
pg_restore: creando CONSTRAINT «employees.dept_manager idx_16889_primary»
pg_restore: creando CONSTRAINT «employees.employees idx_16896_primary»
pg_restore: creando CONSTRAINT «employees.salaries idx_16907_primary»
pg_restore: creando CONSTRAINT «employees.titles idx_16914_primary»
pg_restore: creando INDEX «employees.idx_16877_dept_name»
pg_restore: creando INDEX «employees.idx_16882_dept_no»
pg_restore: creando INDEX «employees.idx_16889_dept_no»
pg_restore: creando FK CONSTRAINT «employees.dept_emp dept_emp_ibfk_1»
pg_restore: creando FK CONSTRAINT «employees.dept_emp dept_emp_ibfk_2»
pg_restore: creando FK CONSTRAINT «employees.dept_manager dept_manager_ibfk_1»
pg_restore: creando FK CONSTRAINT «employees.dept_manager dept_manager_ibfk_2»
pg_restore: creando FK CONSTRAINT «employees.salaries salaries_ibfk_1»
pg_restore: creando FK CONSTRAINT «employees.titles titles_ibfk_1»

```

8.3 Backup binario final

```
pg_dump -h 127.0.0.1 -U marco -d pdb_employees -F c -b -v -f ~/tecBD1/pdb_employees_backup.dump
```

Archivo: `pdb_employees_backup.dump` (~35.9 MB, formato custom `-F c`, con blobs `-b`, verbose `-v`). Contiene esquema `employees` (datos migrados) + esquema `public` (DDL Act. 5) + 2 vistas.

8.4 Validación del backup con `pg_restore -l`

Para demostrar que el backup contiene todo lo exigido, se listó su índice (TOC) el 29/09/2026 (`verificacion_backup.txt`):

```
pg_restore -l ~/tecBD1/pdb_employees_backup.dump > verificacion_backup.txt
```

```

; Archive created at 2026-09-26 17:25:45 -04
;   dbname: pdb_employees
;   TOC Entries: 62
;   Compression: gzip
;   Dump Version: 1.16-0
;   Format: CUSTOM
;
; Selected TOC Entries:
;
6; 2615 24585 SCHEMA - employees marco
868; 1247 16695 TYPE public gender_enum marco
228; 1259 24586 TABLE employees departments marco
229; 1259 24591 TABLE employees dept_emp marco
234; 1259 24674 VIEW employees current_dept_emp marco
235; 1259 24678 VIEW employees dept_emp_latest_date marco
230; 1259 24598 TABLE employees dept_manager marco
231; 1259 24605 TABLE employees employees marco
232; 1259 24616 TABLE employees salaries marco
233; 1259 24623 TABLE employees titles marco
220; 1259 16699 TABLE public departments marco
221; 1259 16708 TABLE public dept_emp marco
226; 1259 24577 VIEW public current_dept_emp marco
227; 1259 24581 VIEW public dept_emp_latest_date marco
222; 1259 16717 TABLE public dept_manager marco
223; 1259 16726 TABLE public employees marco
224; 1259 16737 TABLE public salaries marco
225; 1259 16746 TABLE public titles marco
3550; 0 24586 TABLE DATA employees departments marco
3551; 0 24591 TABLE DATA employees dept_emp marco
3552; 0 24598 TABLE DATA employees dept_manager marco
3553; 0 24605 TABLE DATA employees employees marco

```

```

3554; 0 24616 TABLE DATA employees salaries marco
3555; 0 24623 TABLE DATA employees titles marco
3544; 0 16699 TABLE DATA public departments marco
3545; 0 16708 TABLE DATA public dept_emp marco
3546; 0 16717 TABLE DATA public dept_manager marco
3547; 0 16726 TABLE DATA public employees marco
3548; 0 16737 TABLE DATA public salaries marco
3549; 0 16746 TABLE DATA public titles marco
3368; 2606 24630 CONSTRAINT employees departments idx_16877_primary marco
3371; 2606 24632 CONSTRAINT employees dept_emp idx_16882_primary marco
3374; 2606 24634 CONSTRAINT employees dept_manager idx_16889_primary marco
3376; 2606 24636 CONSTRAINT employees employees idx_16896_primary marco
3378; 2606 24638 CONSTRAINT employees salaries idx_16907_primary marco
3380; 2606 24640 CONSTRAINT employees titles idx_16914_primary marco
3387; 2606 24644 FK CONSTRAINT employees dept_emp dept_emp_ibfk_1 marco
3388; 2606 24649 FK CONSTRAINT employees dept_emp dept_emp_ibfk_2 marco
3389; 2606 24654 FK CONSTRAINT employees dept_manager dept_manager_ibfk_1 marco
3390; 2606 24659 FK CONSTRAINT employees dept_manager dept_manager_ibfk_2 marco
3391; 2606 24664 FK CONSTRAINT employees salaries salaries_ibfk_1 marco
3392; 2606 24669 FK CONSTRAINT employees titles titles_ibfk_1 marco
3381; 2606 16759 FK CONSTRAINT public dept_emp fk_dept_emp_dept marco
3382; 2606 16754 FK CONSTRAINT public dept_emp fk_dept_emp_emp marco
3383; 2606 16769 FK CONSTRAINT public dept_manager fk_dept_mgr_dept marco
3384; 2606 16764 FK CONSTRAINT public dept_manager fk_dept_mgr_emp marco
3385; 2606 16774 FK CONSTRAINT public salaries fk_salaries_emp marco
3386; 2606 16779 FK CONSTRAINT public titles fk_titles_emp marco

```

Lectura: el TOC de **62 entradas** confirma esquema `employees` (6 tablas + datos + 2 vistas + PK/FK `ibfk`) más esquema `public` (DDL Act. 5 + tipo `gender_enum` + FK `fk_*`). El backup es completo y restaurable.

9. Fase 8: script automatizado `ac06.sh`

Para que cualquiera pueda repetir el experimento sin copiar comandos a mano, todo lo anterior se empaqueta en `ac06.sh`: extrae la vista original, recrea las 2 vistas en PostgreSQL, verifica los 6 conteos y genera el `.dump`. Lo que se hace al correr `./ac06.sh | tee migracion.log` es ejecutar esas 4 etapas y guardar la evidencia en `migracion.log`. La salida `SET / CREATE VIEW / CREATE VIEW` más la tabla de conteos es la prueba de que el proceso es automático y auditable.

El archivo `ac06.sh` (estudiante: Marco Antonio Kiataque Uchima) automatiza vistas + verificación + backup:

```

#!/bin/bash
# =====
# Script de Migración y Verificación (MariaDB -> PostgreSQL 18)
# Asignatura: Tecnología de Base de Datos I
# Estudiante: Marco Antonio Kiataque Uchima
# =====

echo "=== 1. Extrayendo vistas originales desde MariaDB ==="
docker exec -it mariadb mariadb -u root -p123456 -D employees -e "SHOW CREATE VIEW current_dept_emp;" >
~/tecBD1/vistas_mariadb.txt

echo "=== 2. Recreando vistas en PostgreSQL (Esquema employees) ==="
psql -h 127.0.0.1 -U marco -d pdb_employees -c "
SET search_path TO employees, public;

CREATE OR REPLACE VIEW current_dept_emp AS
SELECT l.emp_no, d.dept_no, l.from_date, l.to_date
FROM dept_emp l
JOIN departments d ON l.dept_no = d.dept_no
WHERE l.to_date = '9999-01-01';

CREATE OR REPLACE VIEW dept_emp_latest_date AS
SELECT emp_no, MAX(from_date) AS from_date, MAX(to_date) AS to_date

```

```
FROM dept_emp
GROUP BY emp_no;
"

echo "=== 3. Verificando conteo de filas en PostgreSQL ==="
psql -h 127.0.0.1 -U marco -d pdb_employees -c "
SELECT 'employees.employees' AS tabla, COUNT(*) FROM employees.employees
UNION ALL
SELECT 'employees.departments', COUNT(*) FROM employees.departments
UNION ALL
SELECT 'employees.dept_emp', COUNT(*) FROM employees.dept_emp
UNION ALL
SELECT 'employees.dept_manager', COUNT(*) FROM employees.dept_manager
UNION ALL
SELECT 'employees.salaries', COUNT(*) FROM employees.salaries
UNION ALL
SELECT 'employees.titles', COUNT(*) FROM employees.titles;
"

echo "=== 4. Generando Backup Binario (.dump) ==="
pg_dump -h 127.0.0.1 -U marco -d pdb_employees -F c -b -v -f ~/tecBD1/pdb_employees_backup.dump

echo "=== Proceso completado exitosamente ==="
```

Ejecución (./ac06_migracion.sh | tee ~/tecBD1/migracion.log):

```
=== 1. Extrayendo vistas originales desde MariaDB ===
=== 2. Recreando vistas en PostgreSQL (Esquema employees) ===
Contraseña para usuario marco:
SET
CREATE VIEW
CREATE VIEW
=== 3. Verificando conteo de filas en PostgreSQL ===
Contraseña para usuario marco:
      tabla      | count
-----+-----
employees.employees | 300024
employees.departments | 9
employees.dept_emp | 331603
employees.dept_manager | 24
employees.salaries | 2844047
employees.titles | 443308
(6 filas)

=== 4. Generando Backup Binario (.dump) ===
=== Proceso completado exitosamente ===
```

10. Incidencias y solución

#	Error observado	Causa	Solución aplicada
1	ERROR 1045 (28000): Access denied for user 'root'@'172.18.0.1' (2 intentos)	Contraseña de root incorrecta (en compose es 123456 , no 123123)	Reintentar con la clave correcta de MARIADB_ROOT_PASSWORD ; tercer intento OK
2	ERROR 1046 (3D000): No database selected al ejecutar select * from pdb_employees;	No se seleccionó BD y pdb_employees es una base PostgreSQL, no una tabla MariaDB	Usar USE employees; + SHOW TABLES; + SELECT * FROM employees LIMIT 10;

#	Error observado	Causa	Solución aplicada
3	bash: error sintáctico cerca del elemento inesperado '(' al pegar el <code>CREATE VIEW</code> con backticks directo en bash	El DDL de MariaDB se pegó en la shell en vez de en <code>mariadb / psql</code>	Ejecutar vía <code>docker exec mariadb mariadb ... -e "SHOW CREATE VIEW ..."</code> y <code>psql -c "..."</code>
4	psql: FATAL: password authentication failed for user "marco" en una verificación	PGPASSWORD no exportada / typo en la clave (123123)	Exportar <code>PGPASSWORD=123123</code> o escribir la clave correcta cuando pide Contraseña:
5	Diferencia de tipos pgloader vs DDL manual (<code>bigint / text</code> vs <code>integer / gender_enum</code>)	<code>CAST type enum to text</code> en <code>migracion.load</code>	Documentado como decisión de migración; el DDL canónico de la Act. 5 (<code>employees_postgres.sql</code>) conserva <code>integer</code> + <code>CREATE TYPE gender_enum</code>
6	ERROR: could not resize shared memory segment ... No space left on device al verificar huérfanos en PostgreSQL	El contenedor Docker solo tiene 64 MB en <code>/dev/shm</code> y la consulta con paralelismo los agota	Re-ejecutar solo esa sesión con <code>SET max_parallel_workers_per_gather = 0; SET work_mem = '4MB';</code> (ver 6.3: sale <code>SET / SET</code> y los 6 ceros). No modifica datos

11. Conclusiones

- La migración completa transfirió **3 919 015 filas (134.9 MB) en ~18 s con 0 errores**, validada por conteos directos en ambos motores (300 024 / 9 / 331 603 / 24 / 2 844 047 / 443 308), CRC `OK` en las 6 tablas y MD5 idénticos en `departments` (`fa8cbd70...`) y `dept_manager` (`1e7bec35...`).
- Se recrearon las 2 vistas sin la sintaxis propietaria MariaDB (`DEFINER` , `ALGORITHM` , backticks) y se probaron con datos reales: `dept_emp_latest_date` 300 024 filas, `current_dept_emp` 240 124 vigentes. Integridad referencial: 0 huérfanos en las 6 relaciones, en ambos SGBD.
- El ciclo `pg_dump -F c` → `pg_restore` deja una BD final `pdb_employees` con esquema `employees` (datos) + `public` (DDL Act. 5) y un backup binario de ~36 MB cuyo TOC de 62 entradas (tablas, datos, 2 vistas, PK/FK) se validó con `pg_restore -l`.
- El script `ac06.sh` + `migracion.log` hacen el proceso repetible y auditable, cumpliendo el criterio Estratégico (documenta cada paso, verifica y explica adaptaciones).

12. Eje transversal y ODS

Eje transversal (según consigna del docente):

- Tecnologías emergentes y adaptabilidad digital:** se usan herramientas modernas (Docker, PostgreSQL 18, pgloader, PgModeler, pgAdmin4) y el estudiante se adapta a un entorno real de migración entre SGBD distintos.
- Investigación y pensamiento crítico:** se investigan las diferencias de tipos y sintaxis (MariaDB vs PostgreSQL), se prueban casteos (`enum` → `text` , `datetime` → `timestampz` , backticks → comillas dobles) y se resuelven incidencias (ERROR 1045/1046) con criterio técnico.

ODS vinculados:

ODS	Descripción	Vinculación con el proyecto
ODS 4: Educación de calidad	Garantizar una educación inclusiva, equitativa y de calidad	El proyecto fomenta el aprendizaje práctico de migración de bases de datos con evidencias reproducibles
ODS 10: Reducción de las desigualdades	Reducir la desigualdad en y entre los países	El uso de herramientas open source (Docker, PostgreSQL, MariaDB, pgloader) democratiza el acceso al conocimiento

13. Repositorio GitHub

Repositorio de entrega: https://github.com/MarcoKiataque27/tecnologia_de_base_de_datos_1.git

Estructura publicada:

```
tecnologia_de_base_de_datos_1/
├── README.md # Portada del repo (Act. 5 + proyecto final)
├── presentacion de proyecto final/
│   ├── Proyecto_Final_Informe.md
│   ├── Proyecto_Final_Informe.pdf
│   ├── Proyecto_Final_Informe.docx
│   ├── docker-compose.yml
│   ├── migracion.load
│   ├── ac06.sh
│   ├── employees_diagrama.png
│   ├── employees_postgres.sql
│   └── pdb_employees_backup.dump
└── actividad-5/
    ├── Actividad_5_Informe.md
    ├── Actividad_5_Informe.pdf
    ├── employees_diagrama.png
    └── employees_postgres.sql
```

Comandos para publicar:

```
git init
git remote add origin https://github.com/MarcoKiataque27/tecnologia_de_base_de_datos_1.git
git add README.md "presentacion de proyecto final" actividad-5
git commit -m "Entregable final: migracion employees MariaDB a PostgreSQL 18 + Act 5"
git branch -M main
git push -u origin main
```

14. Referencias

- PostgreSQL Global Development Group. (2026). *PostgreSQL 18 documentation*. <https://www.postgresql.org/docs/>
 - MariaDB Foundation. (2026). *mariadb-dump overview*. MariaDB Knowledge Base. <https://mariadb.com/kb/en/mariadb-dump/>
 - pgloader Project. (2026). *pgloader reference manual (v3.6)*. <https://pgloader.readthedocs.io/>
 - PgModeler. (2026). *Data modeling tool for PostgreSQL*. <https://pgmodeler.io/>
 - Datacharmer. (2026). *test_db: MySQL employees sample database*. GitHub. https://github.com/datacharmer/test_db
 - Universidad Privada Domingo Savio. (2026). *Material_EntregableFinal_TBDI: guías Bloque 3 y DokuWiki del docente*. Material de clase.
-